

College Library Database

Database Administration Project

Student Details	Details
Student Name	AMRUDESH
Register Number	2422J1142
Department	BACHELOR OF COMPUTER APPLICATIONS
ID	5FE94A81FD44E0690F9465DBCE2922F5
College	GOVERNMENT ARTS AND SCIENCE COLLEGE GUDALUR
Date of Submission	24/09/2026

Project Scope: A relational database for maintaining books, students/members, issue and return transactions, book availability, searching, and overdue tracking.

1. Problem Statement & Objectives

Problem Statement

A college library handles many books and student members. If book details, member details, and issue/return records are maintained manually, it becomes difficult to know which books are available, who has borrowed a book, when it is due, and which books are overdue. The proposed database centralizes these records and provides structured SQL operations for reliable retrieval and management.

Objectives

- Maintain complete book records including title, author, category, ISBN, and copy counts.
- Maintain student/member records.
- Record every book issue and return transaction.
- Track available copies and borrowing status.
- Search books by title, author, category, or ISBN.
- Identify overdue books using due date and return status.
- Use SQL constraints and relationships to preserve data integrity.

2. Expected Features

Feature	What the database provides
Book records	Stores book ID, title, author, category, ISBN and copy information.
Student/member records	Stores member ID, name, course, semester, phone and email.
Issue and return transactions	Stores issue date, due date, return date and transaction status.
Book availability	Shows total copies and currently available copies.
Search books	Supports partial searches using LIKE and filters using WHERE.
Overdue tracking	Finds active transactions whose due date has passed.
Reporting queries	Uses COUNT, GROUP BY and INNER JOIN for useful summaries.

These features directly match the College Library Database problem statement shown in the provided project prompt.

3. Database Design – Overview

Database name: college_library

The design separates independent entities into related tables. This reduces unnecessary duplication and makes issue/return transactions traceable to both a member and a book.

Table	Purpose	Primary Key
books	Stores book master data and availability.	book_id
members	Stores student/member information.	member_id
transactions	Stores issue and return activity.	transaction_id

Relationship logic

- One member can have many borrowing transactions.
- One book can appear in many transactions over time.
- Each transaction belongs to exactly one member and one book.
- Foreign keys connect transactions to the books and members tables.

4. Table Structures

4.1 books

Column	Type	Constraint	Purpose
book_id	INT	PK, AUTO_INCREMENT	Unique book record
title	VARCHAR(150)	NOT NULL	Book title
author	VARCHAR(100)	NOT NULL	Author name
category	VARCHAR(60)	NOT NULL	Subject/category
isbn	VARCHAR(20)	UNIQUE	ISBN number
total_copies	INT	NOT NULL, CHECK > 0	Total copies
available_copies	INT	NOT NULL, CHECK >= 0	Copies available now

4.2 members

Column	Type	Constraint	Purpose
member_id	INT	PK, AUTO_INCREMENT	Unique member
name	VARCHAR(100)	NOT NULL	Student name
course	VARCHAR(50)	NOT NULL	Course
semester	INT	CHECK 1–8	Semester
phone	VARCHAR(15)	UNIQUE	Contact
email	VARCHAR(120)	UNIQUE	Email

5. Table Structures – Transactions

5.1 transactions

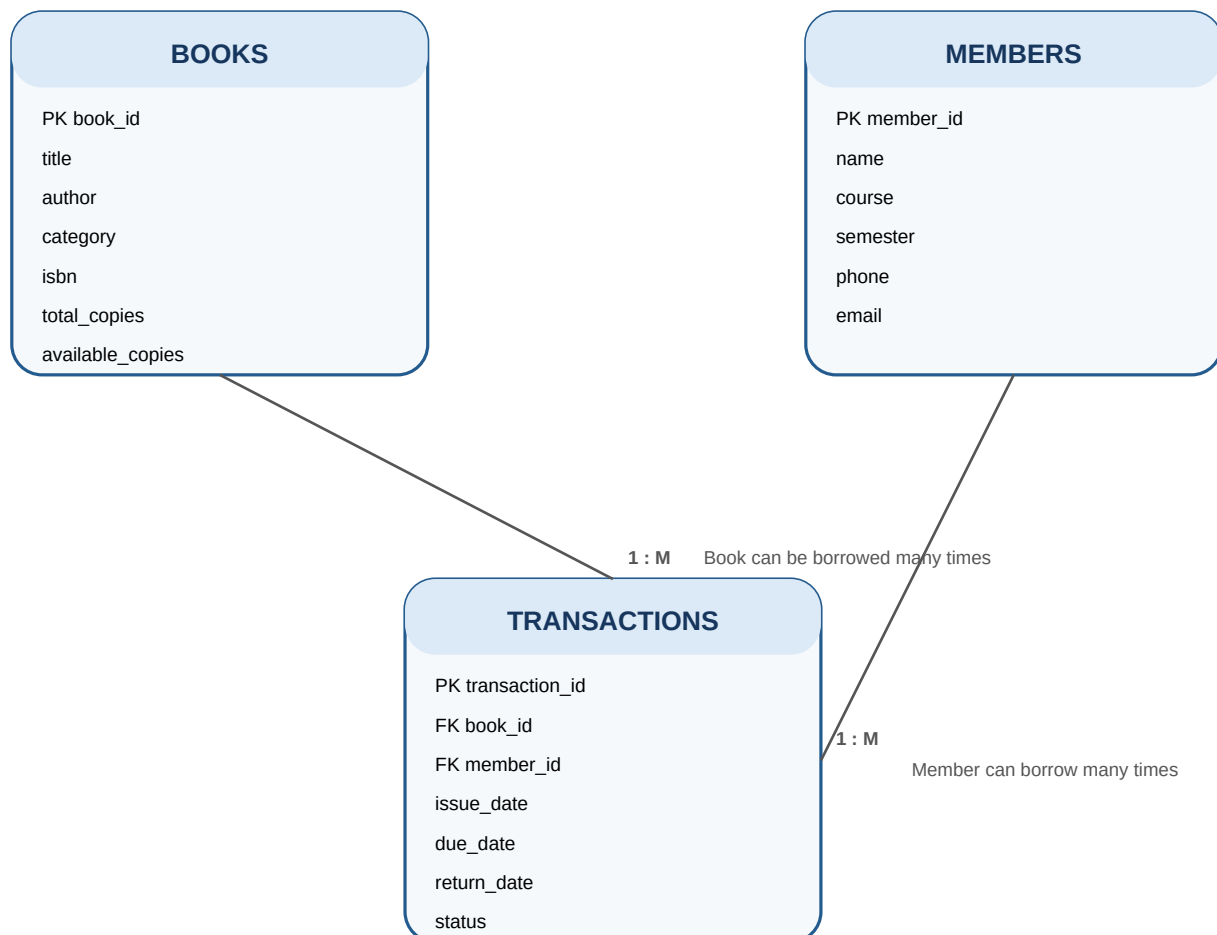
Column	Type	Constraint	Purpose
transaction_id	INT	PK, AUTO_INCREMENT	Unique transaction
book_id	INT	FK → books	Borrowed book
member_id	INT	FK → members	Borrowing member
issue_date	DATE	NOT NULL	Issue date
due_date	DATE	NOT NULL	Expected return date
return_date	DATE	NULL	Actual return date
status	ENUM	NOT NULL	ISSUED / RETURNED

Why separate these tables?

- **books** changes relatively slowly and represents the library's inventory.
- **members** represents people who can borrow books.
- **transactions** represents events. Keeping it separate preserves the borrowing history instead of overwriting it.

6. Entity Relationship Diagram (ERD)

The ERD shows primary keys, foreign keys, and the one-to-many relationships used by the database.



7. Technologies Used

Technology / Tool	Purpose
MySQL	Relational database management system.
MySQL Workbench	Database design, SQL execution and result inspection.
SQL	Database creation, insertion, updates, deletion and queries.
ER Diagram	Visual representation of entities and relationships.

Execution environment

- Create the database in MySQL.
- Run CREATE TABLE statements in MySQL Workbench.
- Insert sample records.
- Execute retrieval, filtering, aggregation and join queries.
- Inspect the result grids and database structure.

8. Practical Implementation – Database Creation

8.1 CREATE DATABASE

```
CREATE DATABASE college_library;  
USE college_library;
```

8.2 CREATE TABLE – books

```
CREATE TABLE books (  
  book_id INT PRIMARY KEY AUTO_INCREMENT,  
  title VARCHAR(150) NOT NULL,  
  author VARCHAR(100) NOT NULL,  
  category VARCHAR(60) NOT NULL,  
  isbn VARCHAR(20) UNIQUE,  
  total_copies INT NOT NULL CHECK (total_copies > 0),  
  available_copies INT NOT NULL CHECK (available_copies >= 0)  
);
```

9. Practical Implementation – Table Creation

9.1 CREATE TABLE – members

```
CREATE TABLE members (  
  member_id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL,  
  course VARCHAR(50) NOT NULL,  
  semester INT NOT NULL CHECK (semester BETWEEN 1 AND 8),  
  phone VARCHAR(15) UNIQUE,  
  email VARCHAR(120) UNIQUE  
);
```

9.2 CREATE TABLE – transactions

```
CREATE TABLE transactions (  
  transaction_id INT PRIMARY KEY AUTO_INCREMENT,  
  book_id INT NOT NULL,  
  member_id INT NOT NULL,  
  issue_date DATE NOT NULL,  
  due_date DATE NOT NULL,  
  return_date DATE,  
  status ENUM('ISSUED', 'RETURNED') NOT NULL DEFAULT 'ISSUED',  
  FOREIGN KEY (book_id) REFERENCES books(book_id),  
  FOREIGN KEY (member_id) REFERENCES members(member_id)  
);
```

10. Data Insertion

10.1 Sample book records

```
INSERT INTO books
(title, author, category, isbn, total_copies, available_copies)
VALUES
('Python Programming', 'Mark Lutz', 'Programming', '9781449355739', 5, 4),
('Database System Concepts', 'Silberschatz', 'Database', '9780078022159', 4, 4),
('Clean Code', 'Robert C. Martin', 'Programming', '9780132350884', 3, 2),
('Data Science from Scratch', 'Joel Grus', 'Data Science', '9781492041139', 2, 2);
```

10.2 Sample member records

```
INSERT INTO members
(name, course, semester, phone, email)
VALUES
('Arun Kumar', 'BCA', 5, '9876543210', 'arun@example.com'),
('Meera S', 'BCA', 5, '9876543211', 'meera@example.com'),
('Rahul P', 'BCA', 3, '9876543212', 'rahul@example.com');
```

11. Issue & Return Transaction Data

11.1 Sample transactions

```
INSERT INTO transactions
(book_id, member_id, issue_date, due_date, return_date, status)
VALUES
(1, 1, '2026-09-10', '2026-09-24', NULL, 'ISSUED'),
(3, 2, '2026-09-12', '2026-09-26', NULL, 'ISSUED'),
(2, 3, '2026-09-01', '2026-09-15', '2026-09-14', 'RETURNED');
```

Important design point: a returned book keeps its transaction history. The return_date and status are updated rather than deleting the transaction.

11.2 Example return operation

```
UPDATE transactions
SET return_date = CURRENT_DATE,
status = 'RETURNED'
WHERE transaction_id = 1;
```

12. SQL Operations – INSERT, SELECT, UPDATE, DELETE

INSERT – add a new member

```
INSERT INTO members
(name, course, semester, phone, email)
VALUES ('Anjali R', 'BCA', 5, '9876543213', 'anjali@example.com');
```

SELECT – retrieve books

```
SELECT book_id, title, author, available_copies
FROM books;
```

UPDATE – modify a member

```
UPDATE members
SET phone = '9999999999'
WHERE member_id = 1;
```

DELETE – remove a member record

```
DELETE FROM members
WHERE member_id = 4;
```

In a real system, deletion should be restricted when the member has dependent transaction records; foreign-key rules help protect referential integrity.

13. SQL Operations – Filtering & Searching

WHERE – filter by category

```
SELECT *  
FROM books  
WHERE category = 'Programming';
```

ORDER BY – sort books alphabetically

```
SELECT title, author  
FROM books  
ORDER BY title ASC;
```

LIKE – partial book search

```
SELECT book_id, title, author  
FROM books  
WHERE title LIKE '%Python%';
```

BETWEEN – books with a chosen copy range

```
SELECT title, total_copies  
FROM books  
WHERE total_copies BETWEEN 2 AND 5;
```

14. Aggregate Functions

Aggregate functions summarize multiple rows into useful values.

```
SELECT COUNT(*) AS total_books
FROM books;

SELECT SUM(total_copies) AS total_copies
FROM books;

SELECT AVG(total_copies) AS average_copies
FROM books;

SELECT MAX(total_copies) AS maximum_copies
FROM books;

SELECT MIN(total_copies) AS minimum_copies
FROM books;
```

Example interpretation

Function	Use in library database
COUNT()	Count book records or transactions.
SUM()	Find total physical copies.
AVG()	Find average copies per title.
MAX()	Find the largest copy count.
MIN()	Find the smallest copy count.

15. GROUP BY – Library Reports

15.1 Count books by category

```
SELECT category, COUNT(*) AS number_of_titles
FROM books
GROUP BY category
ORDER BY number_of_titles DESC;
```

15.2 Count transactions by status

```
SELECT status, COUNT(*) AS transaction_count
FROM transactions
GROUP BY status;
```

GROUP BY converts detailed rows into category-level summaries. It is useful for reports such as number of titles by subject and number of active versus returned transactions.

16. INNER JOIN – Related Data

An INNER JOIN combines matching rows from multiple related tables.

```
SELECT
  t.transaction_id,
  m.name AS member_name,
  b.title AS book_title,
  t.issue_date,
  t.due_date,
  t.status
FROM transactions t
  INNER JOIN members m
    ON t.member_id = m.member_id
  INNER JOIN books b
    ON t.book_id = b.book_id;
```

This query produces a human-readable borrowing list without duplicating member and book information inside the transaction table.

17. Book Availability

A simple availability query can show which titles currently have copies available.

```
SELECT
book_id,
title,
total_copies,
available_copies,
CASE
WHEN available_copies > 0 THEN 'AVAILABLE'
ELSE 'NOT AVAILABLE'
END AS availability
FROM books;
```

Example output

Book	Total	Available	Availability
Python Programming	5	4	AVAILABLE
Database System Concepts	4	4	AVAILABLE
Clean Code	3	2	AVAILABLE
Data Science from Scratch	2	2	AVAILABLE

18. Search Books

18.1 Search by title

```
SELECT book_id, title, author
FROM books
WHERE title LIKE '%Data%';
```

18.2 Search by author

```
SELECT book_id, title, author
FROM books
WHERE author LIKE '%Martin%';
```

18.3 Search by category

```
SELECT book_id, title, category
FROM books
WHERE category = 'Database';
```

LIKE is useful when the user knows only part of the title or author name.

19. Overdue Tracking

An overdue transaction is an active issue whose due date is earlier than today's date.

```
SELECT
  t.transaction_id,
  m.name AS member_name,
  b.title AS book_title,
  t.due_date,
  DATEDIFF(CURRENT_DATE, t.due_date) AS days_overdue
FROM transactions t
INNER JOIN members m
  ON t.member_id = m.member_id
INNER JOIN books b
  ON t.book_id = b.book_id
WHERE t.status = 'ISSUED'
AND t.due_date < CURRENT_DATE;
```

This avoids treating returned books as overdue because the query checks status = 'ISSUED'.

20. Issue & Return Workflow

Step	Action	Database effect
1	Search/select a book	Read books table
2	Check availability	available_copies > 0
3	Select member	Read members table
4	Issue book	Insert transaction + reduce availability
5	Return book	Set return_date + status
6	Restore availability	Increase available_copies
7	Check overdue	Compare due_date with current date

A production implementation should perform issue/return updates in a transaction so the transaction record and availability count stay consistent.

21. Keys & Constraints

Constraint	Used in	Purpose
PRIMARY KEY	All three tables	Uniquely identifies each row.
FOREIGN KEY	transactions	Connects each transaction to a valid book and member.
NOT NULL	Important fields	Prevents missing required data.
UNIQUE	ISBN, phone, email	Prevents duplicate identifiers/contact values.
CHECK	copy counts, semester	Restricts invalid numeric values.
ENUM	transaction status	Restricts status to ISSUED or RETURNED.

22. Database Design Explanation

Why not store everything in one table?

A single large table would repeat the same student and book details for every borrowing event. Separate tables reduce duplication and make updates safer.

Example

Poor design	Separated design
Transaction row repeats member name, course, phone and email.	Member details stored once in members.
Transaction row repeats title, author and ISBN.	Book details stored once in books.
History can be difficult to manage.	Each issue/return event gets its own transaction row.

The result is a cleaner relational structure with explicit relationships through foreign keys.

23. Project Execution & Output Screenshots

The provided template expects screenshots of database creation, table records, CRUD operations, filtering, aggregates, GROUP BY, INNER JOIN and final results. For the final submission, capture these directly from MySQL Workbench after executing the supplied SQL.

Screenshot	What to capture
1	college_library database and three tables in the schema navigator.
2	books table with sample records.
3	members table with sample records.
4	transactions table with issue/return records.
5	INSERT / UPDATE / DELETE execution and result.
6	SELECT, WHERE, ORDER BY, LIKE and BETWEEN results.
7	COUNT, SUM, AVG, MAX and MIN results.
8	GROUP BY result.
9	INNER JOIN borrowing report.
10	Final schema/query results overview.

Note: These are screenshot slots/instructions rather than fabricated screenshots. Replace them with your actual MySQL Workbench screenshots.

24. Technical Explanation

Database Design

The database uses three core entities: books, members and transactions. Books and members contain master data, while transactions records borrowing events.

Keys and Constraints

Primary keys uniquely identify rows. Foreign keys ensure that every transaction refers to an existing book and member. NOT NULL, UNIQUE and CHECK constraints reduce invalid data.

SQL Operations

INSERT adds records, SELECT retrieves records, UPDATE changes existing records, and DELETE removes records. WHERE, ORDER BY, LIKE and BETWEEN refine retrieval.

Aggregate Functions and Joins

COUNT, SUM, AVG, MAX and MIN summarize data. INNER JOIN combines related rows so a transaction can be displayed with the corresponding member and book names.

25. Testing Checklist

Test	Expected result
Create database	college_library is created successfully.
Insert book/member	New row appears in the relevant table.
Issue transaction	Transaction is stored with ISSUED status.
Search by title	Matching titles are returned.
Filter by category	Only selected category is returned.
Aggregate query	Correct summary values are returned.
JOIN query	Member and book names appear with transaction data.
Overdue query	Only active overdue transactions appear.
Return transaction	Status becomes RETURNED and return date is stored.

26. Conclusion & Future Enhancements

Conclusion

The College Library Database provides a structured relational solution for managing book records, student/member records, issue and return transactions, availability, searching and overdue tracking. The project demonstrates database creation, table design, keys and constraints, data insertion, CRUD operations, filtering, aggregate functions, GROUP BY and INNER JOIN.

Future Enhancements

- Student/member login and authentication.
- Online book reservation and issue/return requests.
- Automated due-date reminders.
- Detailed borrowing and overdue reports.
- Fine calculation and invoice/receipt generation.
- Librarian/admin dashboard.
- Integration with a web or mobile application.
- Automated database backup and recovery.

END OF PROJECT